



Norwegian University of
Science and Technology

Chap 2 - Intelligent Agents

Massimiliano Ruocco, PhD
Associate Professor, IDI, NTNU



Main Reference

Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 4th Edition, 2021 — Chapter 2 Intelligent Agents

CHAPTER 2

INTELLIGENT AGENTS

What Do We Mean by "AI"?

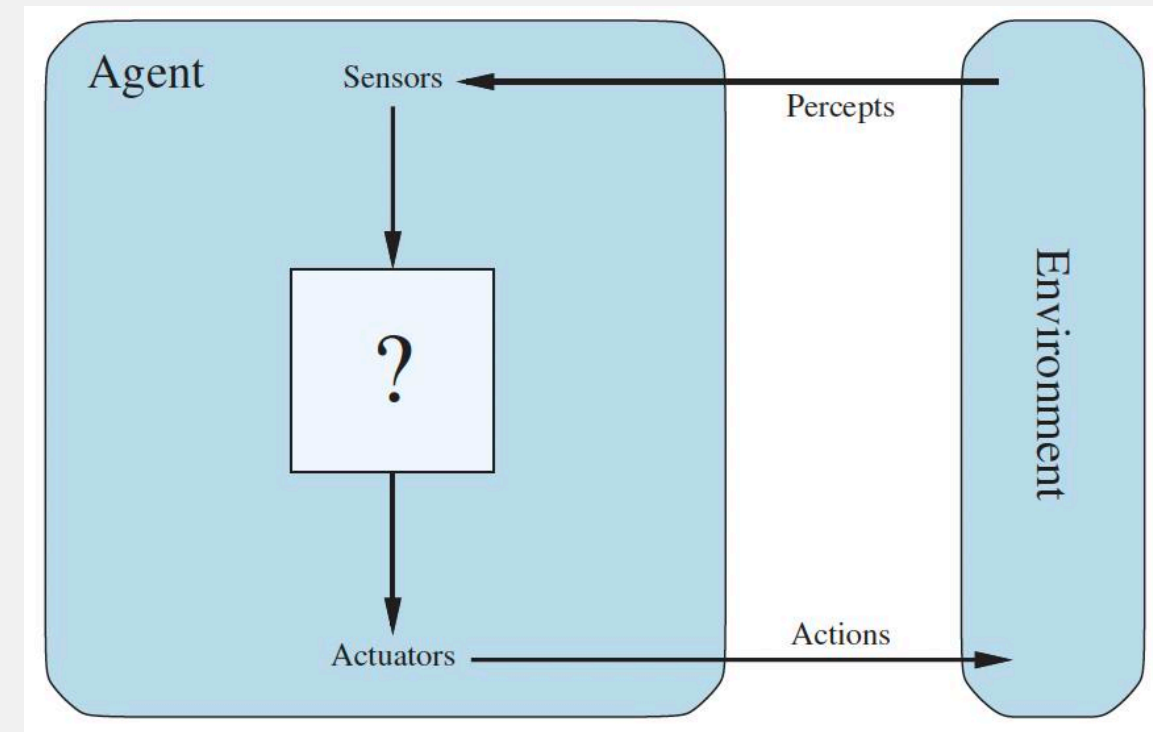
- No single agreed definition of AI — researchers have historically split along two axes: **thought** vs. **behavior**, and **imitating humans** vs. an **ideal** of **rationality**
- This course (and your textbook) follows the acting-rationally / rational-agent approach: an intelligent agent acts to achieve the best expected outcome given what it knows
- Thinking rationally's problem: general deductive inference is computationally intractable, and correct logic alone doesn't automatically produce intelligent behavior
- Acting rationally's two advantages: it isn't tied to one fixed notion of "correct" reasoning, and it's mathematically well-defined enough to actually build and test

John McCarthy (1955): AI is "the science and engineering of making intelligent machines."

	Human-like	Rational
Thinking	Thinking humanly — cognitive modeling	Thinking rationally — "laws of thought"
Acting	Acting humanly — the Turing Test	Acting rationally — the rational agent

What Is an Agent?

- An **agent** is anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that **environment** through **actuators**
- Human agent:
 - eyes, ears, ... (sensors);
 - hands, legs, vocal tract, ... (actuators)
- Robotic agent:
 - cameras, infrared range finders, ... ; motors
- Software agent (softbot):
 - files, network packets, keyboard/mouse input ...;
 - writing files, sending packets, displaying output



The Environment

- The environment could, in principle, be the entire universe
- In practice: just the part of the universe **whose state we care about** when designing this agent
- Specifically: the part that **affects what the agent perceives**, and that is **affected by the agent's actions**
- Typically, the **environment defines the problem**; the **agent provides the solution**

Percepts and Percept Sequences

- **Percept:** the content an agent's sensors are perceiving at a given instant
- **Percept sequence:** the complete history of everything the agent has ever perceived
- An **agent's choice of action** can depend on:
 - its built-in knowledge, and
 - the entire percept sequence observed so far
 - ...but not on anything it hasn't perceived yet

The Agent Function

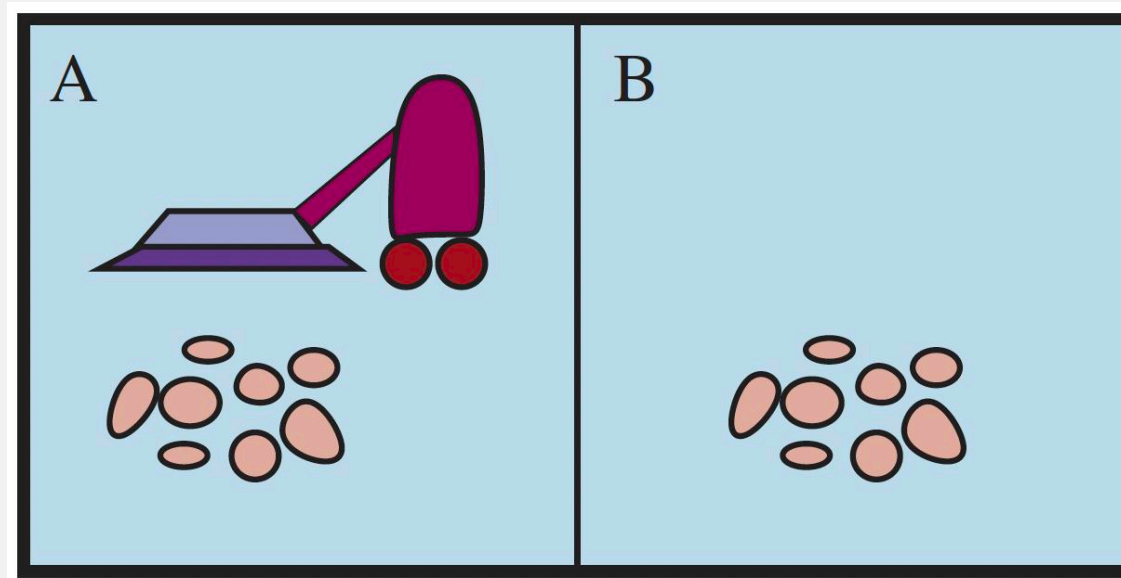
- By specifying the agent's action for every possible percept sequence, we've said everything there is to say about the agent's behavior
- **Agent function:** the mathematical mapping from percept sequences to actions
- This is an external, **abstract characterization** — not how the agent is actually built

Agent Function vs. Agent Program

- **Agent function:** abstract mathematical description of behavior
- **Agent program:** the concrete implementation that runs on the physical agent architecture, and produces that behavior
- **agent = architecture + program**
- We can imagine tabulating the agent function — for most agents, this table would be astronomically large, even infinite if percept sequences are unbounded

Running Example — The Vacuum-Cleaner World

- A robotic vacuum agent in a world of squares that are dirty or clean
- Simplest version: *just two squares, A and B*
- Percepts: *which square it's in, and whether that square is dirty*
- Actions available: *move Right, move Left, Suck, do nothing*
- Agent starts in square A



A Simple Agent Function for the Vacuum World

- One very simple agent function: if the current square is dirty, suck; otherwise, move to the other square
- Partial tabulation:

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A,Clean],[A,Clean]	Right
[A,Clean],[A,Dirty]	Suck
...	...

So... What Makes an Agent Good?

- We've now defined what an agent is
- **Different agent functions produce different behavior** in the same environment — some clearly better than others
- The obvious next question: what makes an agent good or bad, intelligent or stupid?
 - This is exactly the concept of **Rationality**, answers

Good Behavior — The Concept of Rationality



- **Rational agent:** one that "does the right thing"
- AI adopts **consequentialism**: we judge an agent by the consequences of its behavior, not its intentions
- An **agent generates a sequence of actions** → this causes the environment to pass through a sequence of states
- If that sequence of states is desirable, the agent performed well

Performance Measures

- **Performance measure:** an objective function that evaluates any given sequence of environment states
- **Desirability** is captured **externally** — in the **mind of the designer or user**, not (necessarily) inside the agent
- Example — vacuum cleaner: measure by amount of dirt cleaned in an 8-hour shift?
 - A rational agent can then just... clean, dump the dirt back on the floor, clean again, repeat!
 -  What you ask for is what you get.

Rationality Depends on Four Things

What is **rational** at any given moment depends on:

1. The **performance** measure defining success
2. The agent's prior **knowledge** of the **environment**
3. The **actions** the agent can perform (**actuators**)
4. The agent's **percept** sequence to date (**sensors**)

The Definition of a Rational Agent

*For each possible percept sequence, a rational agent should select an action that is **expected to maximize** its performance measure, **given the evidence provided by the percept sequence** and whatever built-in knowledge the agent has.*

Example - Is the Vacuum Agent Rational?

Assume:

- **Performance measure:** 1 point per clean square per time step, over 1000 time steps
- **Geography known a priori** (2 squares); dirt distribution & starting location unknown
- Clean stays clean; Suck cleans the current square; Right/Left move one square (or stay, at the edge)
- **Available actions:** *Right, Left, Suck*; sensors correctly report location and dirt

→ Under these assumptions, yes — this simple agent is rational. Its expected performance is at least as good as any other agent's.

What is **rational** at any given moment depends on:

1. The **performance** measure defining success
2. The agent's prior **knowledge** of the environment
3. The **actions** the agent can perform
4. The agent's **percept** sequence to date

Omniscience, Learning, and Autonomy

Rationality $\neq$ Perfection



- **Omniscient agent:** knows the actual outcome of its actions in advance — impossible in reality
- Example: crossing the street safely, then a cargo door falls off a passing airliner and hits you — were you irrational to cross?
 - No! Rationality maximizes expected performance; perfection maximizes actual performance.
- Our definition only requires **acting sensibly given the percept sequence to date** — not predicting the unpredictable

Learning and Autonomy

- A rational agent should also **learn** as much as possible **from what it perceives**
- Prior knowledge can be **modified** and **augmented** with experience
- **Autonomy**: relying on your own percepts and learning, rather than solely on the designer's built-in knowledge
- Cautionary example of agents that lack autonomy:
 - Dung beetle: keeps "plugging" its nest even after the dung ball is removed — never notices



PEAS Example - The Automated Taxi Driver

- PEAS (**P**erformance, **E**nvironment, **A**ctuators, **S**ensors)

Performance measure	Safe, fast, legal, comfortable trip; maximize profits; minimize impact on other road users
Environment	Roads, other traffic, police, pedestrians, customers, weather
Actuators	Steering, accelerator, brake, signal, horn, display, speech
Sensors	Cameras, radar/lidar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

More PEAS Examples

Agent Type	Performance Measure	Environment	Actuators	Sensors
<i>Satellite image analysis</i>	Correct categorization of objects, terrain	Orbiting satellite, downlink, weather	Display of scene categorization	High-res digital camera
<i>Part-picking robot</i>	% of parts in correct bins	Conveyor belt with parts, bins	Jointed arm and hand	Camera, tactile & joint-angle sensors
<i>Refinery controller</i>	Purity, yield, safety	Refinery, raw materials, operators	Valves, pumps, heaters, stirrers	Temperature, pressure, flow, chemical sensors
<i>Interactive English tutor</i>	Student's score on test	Set of students, testing agency	Display of exercises, feedback, speech	Keyboard entry, voice

Properties of Task Environments - Overview

Environments vary along several key dimensions:

1. Fully vs. partially (or un-)observable
2. Single-agent vs. multiagent
3. Deterministic vs. nondeterministic
4. Episodic vs. sequential
5. Static vs. dynamic
6. Discrete vs. continuous
7. Known vs. unknown

1 - Fully vs. Partially Observable

- **Fully observable:** the agent's sensors give it access to the complete state of the environment at every point in time (or at least everything relevant to the choice of action)
 - Convenient — no need to maintain internal state
 - Example: chess, image analysis
- **Partially observable:** parts of the state are hidden (noisy/inaccurate sensors, or the info just isn't in the percept)
 - Example: poker, taxi driving, medical diagnosis
- If the agent has no sensors at all, the environment is **unobservable**

2 — Single-Agent vs. Multiagent

- **Single-agent:** e.g., an agent solving a crossword puzzle alone
- **Multiagent:** e.g., chess (two-agent)
- **Subtlety:** is entity B best modeled as an agent, or just as an object obeying physics (like waves or wind)?
 - Key test: is B's behavior best described as maximizing a performance measure that depends on A's behavior?
- **Competitive multiagent:** chess — B maximizing its score minimizes A's score
- **Cooperative/competitive** mix: taxi driving — avoiding collisions is cooperative; competing for the same parking space is competitive

3 — Deterministic vs. Nondeterministic

- **Deterministic:** the next state is completely determined by the current state and the agent's action
- **Nondeterministic:** there's genuine uncertainty about the outcome
- A **partially observable environment** can appear nondeterministic even if it "really" isn't
- **Stochastic** = nondeterministic outcomes are explicitly quantified with probabilities (vs. just "there's a chance of X")
- *Examples:* chess (deterministic) · taxi driving, poker (nondeterministic/stochastic)

4 — Episodic vs. Sequential

- **Episodic:** experience is divided into atomic episodes — perceive, then act, once — and the next episode doesn't depend on previous actions
 - Example: an agent spotting defective parts on an assembly line
- **Sequential:** the current decision can affect all future decisions
 - Example: chess, taxi driving
- *Episodic environments are much simpler* — no need to think ahead

5 — Static vs. Dynamic

- **Static:** the environment does not change while the agent is deliberating
 - Example: crossword puzzles
- **Dynamic:** the environment can change while the agent thinks — "not deciding" counts as deciding to do nothing
 - Example: taxi driving
- **Semidynamic:** the world itself doesn't change, but the agent's performance score does as time passes
 - Example: chess with a clock

6 — Discrete vs. Continuous

- Applies to: the **state of the environment**, the **handling of time**, and the **agent's percepts and actions**
- *Chess*: finite number of distinct states, discrete percepts, discrete actions
- *Taxi driving*: continuous state and continuous time — speed and location sweep through continuous values; steering angle is continuous too

7 — Known vs. Unknown

- This describes **the agent/designer's state of knowledge** about the rules, not the environment itself
- **Known**: the outcomes (or outcome probabilities) for all actions are given
- **Unknown**: the agent must learn how the environment works
- Importantly not the same axis as observable/partially-observable:
 - Known but partially observable: solitaire — you know the rules, but can't see face-down cards
 - Unknown but fully observable: a new video game — full screen visible, but you don't know what the buttons do yet

From Behavior to Implementation — Agent Programs

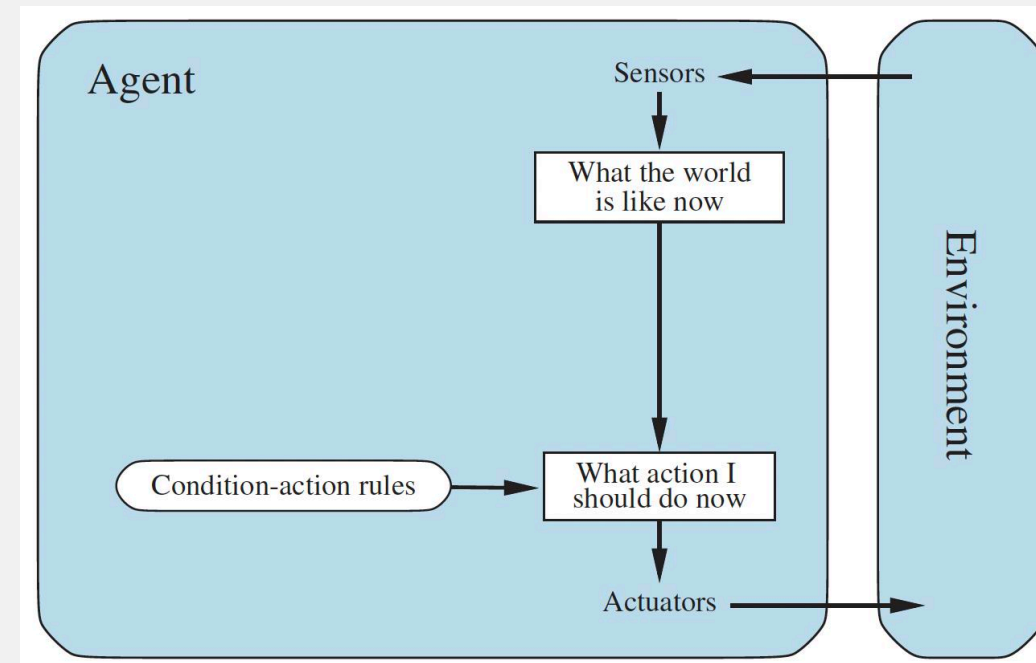
- So far: describing agents by their behavior (agent function)
- Now: how do the insides actually work?
- **Agent program:** implements the agent function, runs on some physical agent architecture
 - agent = architecture + program
- The architecture must fit the program: a program that recommends "Walk" needs an architecture with legs!

1 - Why Not Just... Table-Driven Agents?

- **Table-driven agent:** keeps the full percept sequence in memory, looks up the matching action in a giant table
- It technically does implement any desired agent function correctly...
- ...but is doomed to failure in practice:
 - Automated taxi: $\sim 10^{600,000,000,000}$ table entries for just one hour of driving
 - Chess (a tiny, well-behaved slice of the real world!): at least 10^{150} entries
 - Compare: atoms in the observable universe $\approx 10^{80}$
- **No physical agent has room to store it, no designer has time to build it, and no agent could ever learn it from experience**

2 - Simple Reflex Agents

- **Select actions based only on the current percept**, ignoring the rest of the percept history
- Built from **condition–action rules**: "if car-in-front-is-braking then initiate-braking"
- *Example*: function REFLEX-VACUUM-AGENT([location, status]) returns an action
 - if status = Dirty then return Suck
 - else if location = A then return Right
 - else if location = B then return Left
- **Dramatically smaller** than the full table — but **only works** if the **environment** is **fully observable**

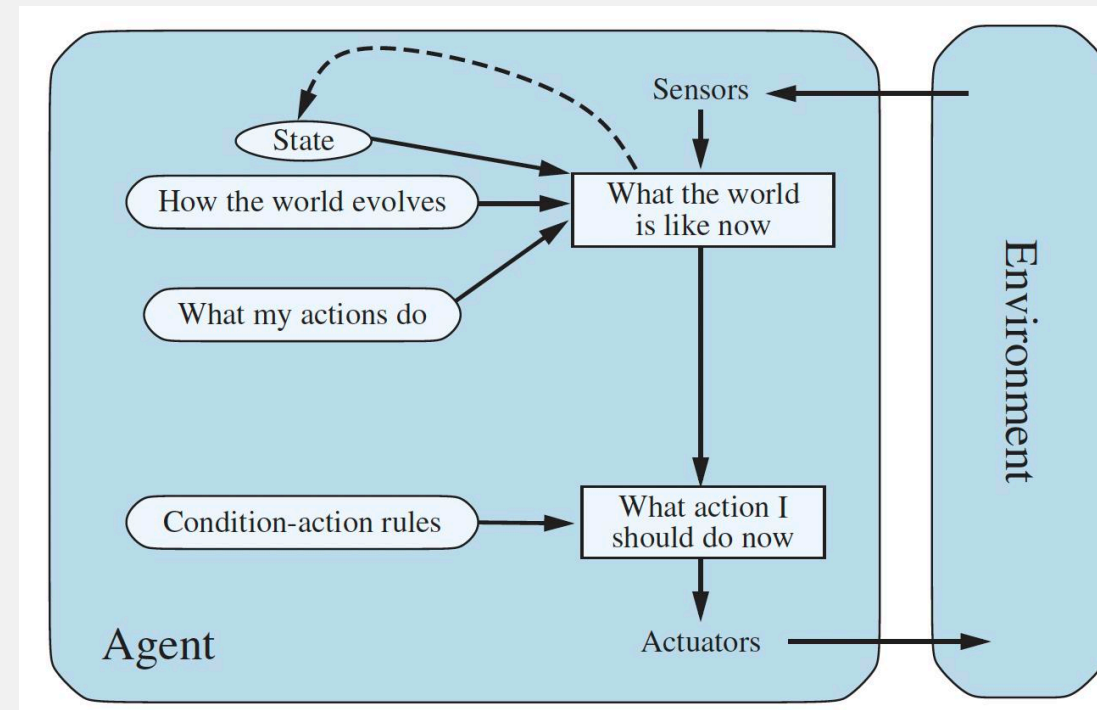


3 - The Limits of Simple Reflex Agents

- **Partial observability** can cause **infinite** loops
 - Vacuum agent with only a dirt sensor (no location sensor): sees [Clean] — should it go left or right? If it always picks the same direction, it can loop forever
- **Randomization** as an escape hatch: flip a coin between Left/Right on [Clean] — reaches the other square in ~ 2 steps on average
- In **single-agent environments**, **randomization** is **rarely optimal** — but it's a useful trick here

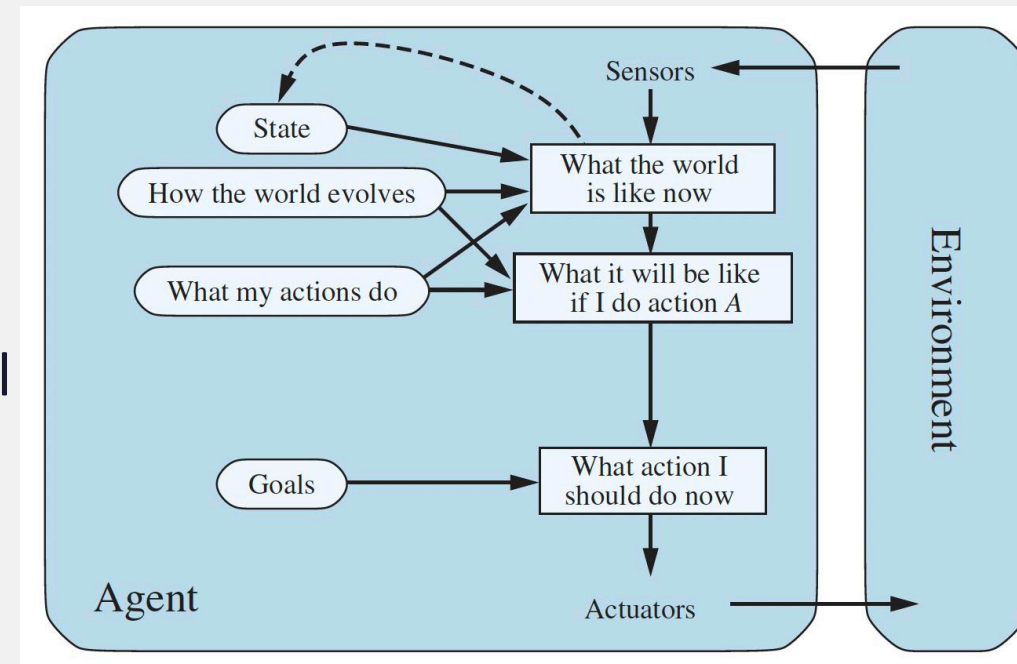
4 - Model-Based Reflex Agents

- **Key idea:** maintain internal state to track parts of the world not visible in the current percept
- Requires two kinds of **built-in knowledge**:
 - Transition model — how the world evolves: effects of the agent's own actions + how the world changes independently
 - Sensor model — how the current world state is reflected in percepts
- Combines **transition model** + **sensor model** + **condition-action rules** to decide the next action
- The agent's **internal state** is really its "**best guess**" about the **world** — full certainty usually isn't achievable in partially observable settings



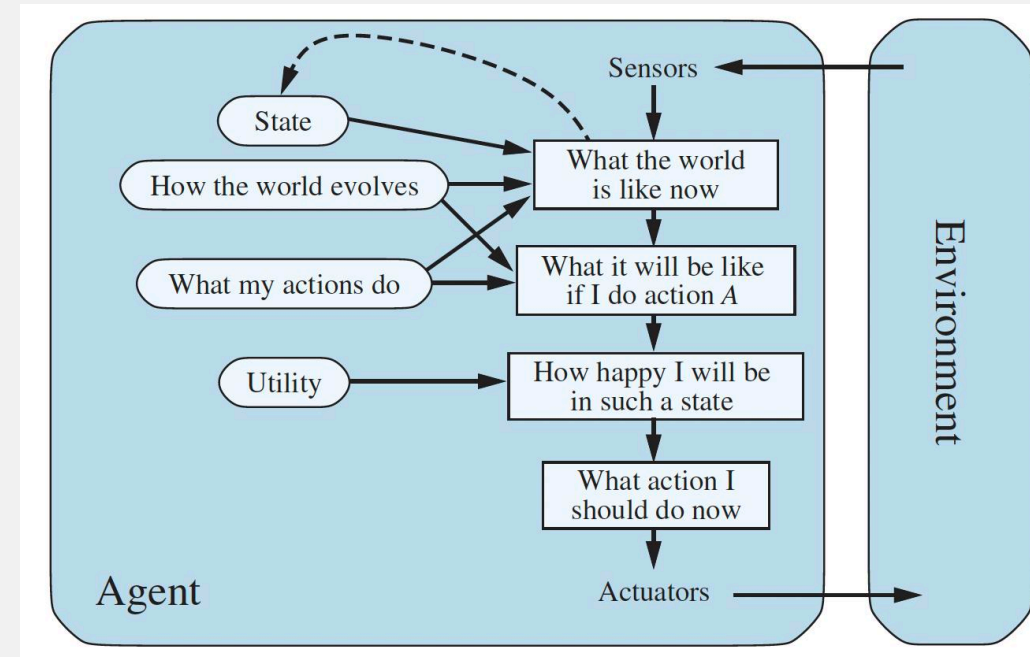
5 - Goal-Based Agents

- Knowing the current state isn't always enough — you also need to know what you're trying to achieve
- **Goal:** explicit information describing desirable situations (e.g., "being at a particular destination")
- **Combines** the model (same as model-based reflex) + goals → chooses actions that will achieve the goal
- Requires considering the **future**: "what will happen if I do this?" and "will that make me happy?"
- **More flexible than reflex agents:** goals are explicit and can be changed — e.g., "go to a different destination" is a one-line change, not a rule rewrite
- This is exactly what search (Ch. 3, 4, 6) and planning (Ch. 11) are for



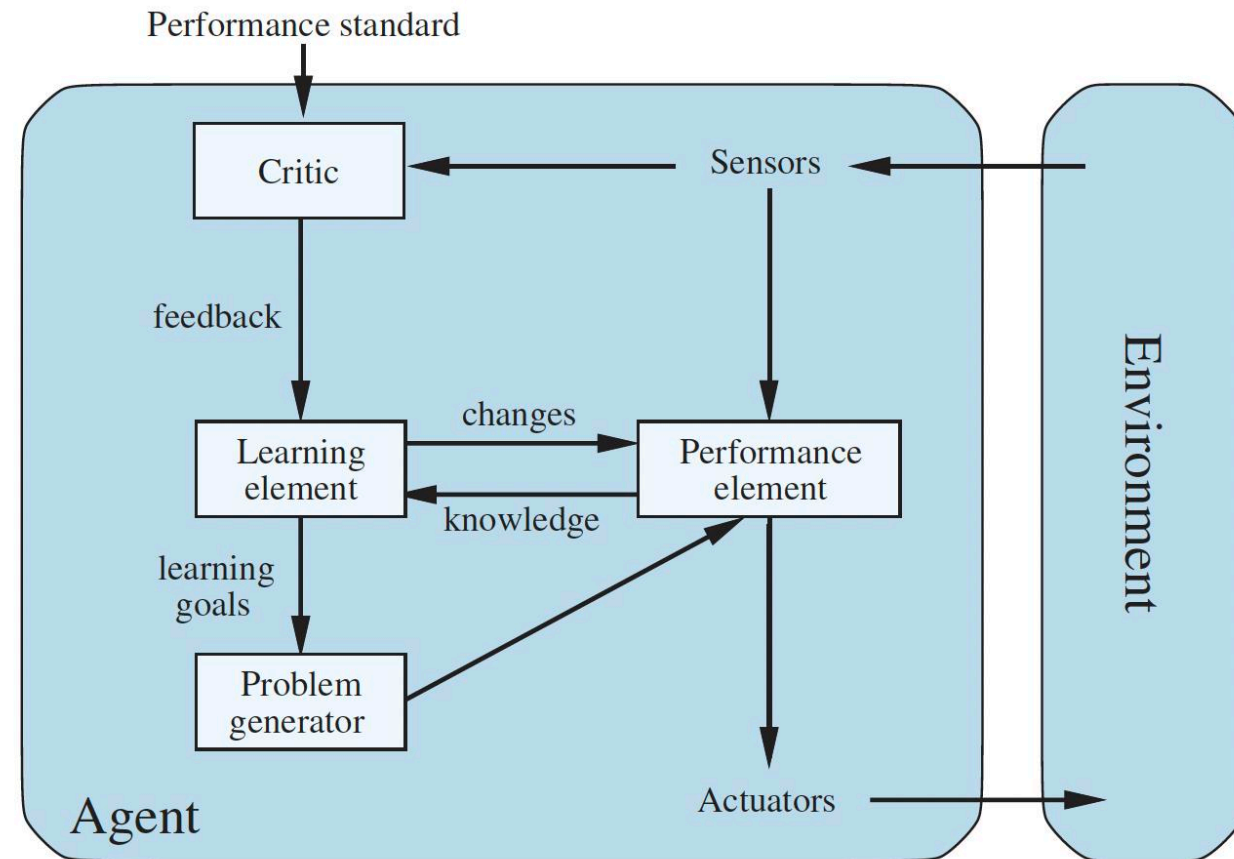
6 - Utility-Based Agents

- Goals give only a **binary distinction**: happy / unhappy
- Some goals can be reached in **multiple ways** with very different quality — need a finer-grained comparison
- Utility function**: maps a state (or sequence of states) onto a real number — an internalization of the performance measure
- A **rational utility-based agent** chooses the action maximizing expected utility: averaging utility over possible outcomes, **weighted by probability**
- Handles **two cases** goals alone can't:
 - Conflicting goals (e.g., speed vs. safety) — utility specifies the right tradeoff
 - Uncertain goal achievement — utility weighs likelihood of success against the goal's importance



Learning Agents

- Any of the previous designs (model-based, goal-based, utility-based) can be built as a **learning agent**
- **Four conceptual components:**
 - Performance element — what we've previously called "the whole agent": selects external actions
 - Critic — tells the learning element how the agent is doing, using a fixed external performance standard
 - Learning element — makes improvements to the performance element based on critic feedback
 - Problem generator — suggests exploratory actions to gather new, informative experience (even if suboptimal short-term)
- Lets an agent start in an unknown environment and become genuinely competent over time



Summary

- An agent perceives and acts; **behavior = agent function**; **rationality = maximizing expected performance** given percepts and prior knowledge
- Task environments are specified via PEAS, and classified along 7 key dimensions — this classification drives agent design choices
- Agent programs, in increasing sophistication:
 - Table-driven → Simple reflex → Model-based reflex → Goal-based → Utility-based
 - ...and any of these can be made a learning agent
- Next lecture: Chapter 3 — Problem-Solving as Search.